

Homework 3

Name: Diya Jain U81673988

Answer 1:

Let $T(n)$ be the worst-case time complexity of the RecursionMystery algorithm for an array of size n . The algorithm splits the array into four parts and recursively calls itself on two concatenated parts. Let's denote the size of the array as n . Each recursive call splits the array into at most 4 parts and concatenates 2 adjacent parts. Therefore, the size of the subproblem reduces to $n/2$ in each recursive call.

The recurrence relation can be expressed as follows:

$$T(n) = 4T\left(\frac{n}{2}\right) + O(n)$$

The $4T\left(\frac{n}{2}\right)$ term accounts for the four recursive calls on arrays of size $n/2$.

The $O(n)$ term accounts for the concatenation operation.

Answer 2:

$$\begin{array}{c} T(n) \\ / \quad \backslash \\ T(n/2) \quad T(n/2) \\ / \quad \backslash \quad / \quad \backslash \\ T(n/4) \quad T(n/4) \dots \end{array}$$

At each level, there are 4 subproblems, and each subproblem has size $n/2$. This branching continues until the subproblems reach a base case of size 1. The number of levels in the tree is $\log_2 n$ as each level halves the size of the input.

Now, summing up the work across all levels:

$$\sum_{i=0}^{\log_2 n} O(n) \times 4^i$$

Using the geometric series formula, we can simplify this to

$$\begin{aligned} & \sum_{i=0}^{\log_2 n} O(n) \times 4^i \\ &= O(n) \times \frac{4^{\log_2 n + 1} - 1}{4 - 1} \\ &= O(n) \times \frac{4n - 1}{3} \\ &= O(n \log n) \end{aligned}$$

Answer 3:

To determine the largest number of recursive calls of $\frac{n}{2} \times \frac{n}{2}$ matrix multiplications that Dr. Adirolf's algorithm can make while still being faster than the ordinary $\Theta(n^3)$ algorithm for matrix multiplication, we need to find the crossover point where Dr. Adirolf's algorithm becomes more efficient.

In Dr. Adirolf's algorithm, each recursive call results in four subproblems of size $\frac{n}{2} \times \frac{n}{2}$. Therefore, the number of recursive calls, $T(n)$, can be described by the recurrence relation:

$$T(n) = 4T\left(\frac{n}{2}\right) + \theta(n^2)$$

In the ordinary $\theta(n^3)$ algorithm for matrix multiplication, the time complexity is $\theta(n^3)$.

To find the crossover point, we set the two complexities equal:

$$4T\left(\frac{n}{2}\right) + \theta(n^2) = \theta(n^3)$$

Solving this recurrence relation, we'll find the point where Dr. Adirolf's algorithm becomes more efficient.

Let's denote $m = \frac{n}{2}$. Then, the recurrence becomes:

$$T(m) = 4T\left(\frac{m}{2}\right) + \theta(m^2)$$

Now, we can use the Master Theorem to solve this recurrence relation.

The Master Theorem states that if $T(n) = aT\left(\frac{n}{b}\right) + f(n)$, according to that $a = 4$, $b = 2$ and $f(n) = \theta(n^2)$. So $\log_b a = 2$.

Because $f(n) = \theta(n^{\log_b a})$

$$T(n) = \theta(n^2 \log n).$$

Now, we compare this with the $\theta(n^3)$ complexity of the ordinary algorithm.

For Dr. Adirolf's algorithm to be faster, we need:

$$n^2 \log n < n^3$$

$$\log n < n$$

The inequality holds true for $n > 1$

Therefore, Dr. Adirolf's algorithm becomes faster than the ordinary $\theta(n^3)$ algorithm for matrix multiplication when $n > 1$.

The largest number of recursive calls of $\frac{n}{2} \times \frac{n}{2}$ matrix multiplications that Dr. Adirolf's algorithm can make while still being faster than the ordinary $\theta(n^3)$ algorithm is $\log n$.

Answer 4:

(a) To find the median of an array of size n , there are multiple ways an algorithm can approach it, but all would essentially aim to find the index of the middle element in the sorted order. The middle element would be the $\frac{n+1}{2} - th$ smallest element in the array. Therefore, the number of different possible solutions for an algorithm

that returns the index of the median of an array of size n is 1. This is because regardless of the specific algorithm used, the median is always the middle element in the sorted array, so there's only one correct index.

(b) The minimum number of comparisons that a comparison-based median algorithm would need to make is $\log_2(n!)$. This is because there are $n!$ possible permutations of the array, and each comparison can divide the number of possibilities by at most 2.

For example, let's consider a binary search-like algorithm that divides the array into two parts based on comparisons. At each step, it compares the element with the current guess for the median. If the element is smaller, it focuses on the upper half; if it's larger, it focuses on the lower half. This process continues recursively until the median is found.

In the worst-case scenario, each comparison divides the number of possible solutions by 2. To ensure that all possible permutations are accounted for, the number of comparisons needed is $\log_2(n!)$.

So, the minimum number of comparisons is $\log_2(n!)$.